JSON & JSON Schema Validation

Student Lab Documentation — Labs 07 & 08

DSS-L3-CS

Prepared by: Boudjellal Bochra

Topics Covered

XML vs JSON • JSON Data Types • JSON Structures
DTD / XSD vs JSON Schema • JSON Schema Validation

1. From XML to JSON — Understanding the Shift

In the previous labs you worked with XML and its validation mechanisms: DTD and XSD. JSON was designed to solve many of the verbosity and complexity problems of XML while keeping the same goal: structured, portable data. This section compares both technologies so you can clearly see where JSON fits.

1.1 The XML Ecosystem You Already Know

Before JSON, the standard XML workflow looked like this:

Technology	Role	What you learned
XML	Data format	Markup language with opening/closing tags
DTD	Validation	Declares elements, attributes, structure (older standard)
XSD	Validation	Stronger schema with types, constraints, namespaces

JSON introduces a parallel ecosystem that achieves the same goals with much less syntax:

Technology	Role	Equivalent in XML world
JSON	Data format	XML
JSON Schema	Validation	XSD (closer) or DTD (simpler)

1.2 XML vs JSON — Syntax Comparison

The same data — a library book — expressed in both formats:

XML

```
<?xml version="1.0"?>
<book>
  <title>Clean Code</title>
  <author>R. C. Martin</author>
  <year>2008</year>
  <available>true</available>
  <keywords>
    <keyword>programming</keyword>
    <keyword>refactoring</keyword>
  </keywords>
</book>
```

JSON

```
{
  "title": "Clean Code",
  "author": "R. C. Martin",
  "year": 2008,
  "available": true,
  "keywords": [
    "programming",
    "refactoring"
  ]
}
```

1.3 Detailed Feature Comparison

Feature	XML	JSON
Syntax	Verbose — opening + closing tags	Compact — key: value pairs

Feature	XML	JSON
Data types	Everything is text by default	Native: string, number, boolean, null, object, array
Arrays	Repeated elements (no dedicated syntax)	Native array syntax [ ]
Boolean	<available>true</available>	"available": true
Numbers	<year>2008</year> (text!)	"year": 2008 (real number)
Comments	<!-- comment --> supported	Not supported
Attributes	<book id="1">	No attributes — everything is a key
Namespaces	Supported (complex)	Not supported (not needed)
Readability	Moderate	High
File size	Larger (tag repetition)	Smaller
Parse speed	Slower	Faster
Use in APIs	Less common today	Dominant standard

1.4 Validation: DTD / XSD vs JSON Schema

Just as XML documents are validated against DTD or XSD, JSON documents are validated against a JSON Schema. Here is how the concepts map to each other:

Concept	DTD	XSD	JSON Schema
Define a field	<!ELEMENT>	<xs:element>	"properties": { ... }
Require a field	implied (presence)	use="required"	"required": ["field"]
Data type	Not supported	<xs:string>, <xs:int>	"type": "string"
Restrict values	Not supported	<xs:enumeration>	"enum": [...]
String length	Not supported	minLength, maxLength	"minLength": 2
Number range	Not supported	minInclusive	"minimum": 0
Array / repetition	<!ELEMENT list (item*)>	maxOccurs	"type": "array", "items": {...}
Nested structure	Nested elements	Nested complexType	"type": "object", "properties": {...}
Schema file format	.dtd file	.xsd file (XML)	.json file (JSON)

Key Insight

JSON Schema is the closest equivalent to XSD — both support data types, value constraints, required fields, and nested structures. DTD is older and weaker (no types, no ranges).
If you understood XSD, JSON Schema will feel very natural.

2. JSON Fundamentals

2.1 What is JSON?

JSON (JavaScript Object Notation) is a lightweight, text-based format for storing and exchanging structured data. It is language-agnostic and is the dominant data format used in web APIs, configuration files, and databases today.

2.2 The 6 JSON Data Types

Type	Example	Notes
string	"Hello World"	Text — always in double quotes
number	42 / 3.14	Integer or decimal — no quotes
boolean	true / false	Lowercase only — not True or False
null	null	Represents an absent or empty value
object	{ "key": value }	Unordered set of key-value pairs
array	[1, "a", true]	Ordered list of any values

2.3 JSON Syntax Rules

- Keys must be strings enclosed in double quotes.
- Values can be any of the 6 types above.
- Key-value pairs are separated by commas.
- Objects are wrapped in curly braces { }.
- Arrays are wrapped in square brackets [ ].
- No trailing commas are allowed.
- No comments are allowed in JSON.

2.4 Common Syntax Errors

Error	Wrong	Correct
Missing quotes on key	{ name: "Alice" }	{ "name": "Alice" }
Single quotes	{ 'name': 'Alice' }	{ "name": "Alice" }
Trailing comma	{ "a": 1, }	{ "a": 1 }
Wrong boolean casing	{ "ok": True }	{ "ok": true }
Number as string	{ "year": "2020" }	{ "year": 2020 }
Comment in JSON	// comment { "a": 1 }	Remove — comments not supported

3. Lab 07 – Introduction to JSON

Objective: Manipulate JSON structures.

Part 1 – Convert the Library XML Document into JSON

You are given an XML document representing a library. Your task is to convert it into a valid JSON document.

Conversion Rules

- The root XML element becomes a JSON object.
- Repeated sibling elements (like multiple <book>) become a JSON array.
- Child elements become key-value pairs inside a JSON object.
- Tag names become string keys (always in double quotes).
- Text content becomes the value — use the correct JSON type (number, string…).

Starting XML

```
<?xml version="1.0" encoding="UTF-8"?>
<library>
  <book>
    <id>1</id>
    <title>The Great Gatsby</title>
    <author>F. Scott Fitzgerald</author>
    <year>1925</year>
    <genre>Fiction</genre>
  </book>
  <book>
    <id>2</id>
    <title>Clean Code</title>
    <author>Robert C. Martin</author>
    <year>2008</year>
    <genre>Technology</genre>
  </book>
</library>
```

Your Task

Convert the XML document above into a JSON document.
Pay attention to: correct data types (id and year should be numbers, not strings),
and the structure — the two books should be inside an array.

Part 2 – Add a Boolean Field: available

A boolean field holds only true or false. Add an "available" field to each book in your JSON document to indicate whether that book is currently borrowable.

Reminder

Boolean values in JSON are: true or false (lowercase, no quotes).
"available": true ✓ correct

"available": "true" ✗ wrong (this would be a string, not a boolean)

Part 3 – Add an Array of Keywords

Add a "keywords" field to each book. This field should be an array of strings representing themes or topics of the book.

Reminder

Arrays use square brackets: "keywords": ["word1", "word2", "word3"]
Items inside an array are separated by commas.
Each string item must be in double quotes.

4. Lab 08 – JSON Schema Validation

Objective: Validate JSON documents using JSON Schema.

4.1 What is JSON Schema?

JSON Schema is a standard that describes the structure and constraints of a JSON document. It plays the same role as XSD for XML: it defines what a valid document must look like, which fields are required, what types they must have, and what values are allowed.

Analogy

XML → validated by → XSD (or DTD)
JSON → validated by → JSON Schema

4.2 Essential JSON Schema Keywords

Keyword	Purpose	Example value
$schema	Declares the JSON Schema version	"http://json-schema.org/draft-07/schema#"
title	Human-readable name for the schema	"Digital CV"
type	Specifies the data type	"string" / "number" / "object" / "array"
properties	Defines the fields of an object	{ "name": { "type": "string" } }
required	Lists mandatory fields	["firstName", "email"]
minimum	Minimum value for a number	0
minLength	Minimum number of characters	2
maxLength	Maximum number of characters	100
items	Schema for each element of an array	{ "type": "string" }
minItems	Minimum number of items in an array	1
enum	List of allowed values	["Excellent","Very Good","Good"]
format	Semantic format check	"email" / "date"
description	Explains the purpose of a field	"User email address"

4.3 JSON Schema Structure

Every JSON Schema document follows this skeleton:

```
{
```

```
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "Name of your schema",
  "type": "object",
  "required": [ "field1", "field2" ],
  "properties": {
    "field1": {
      "type": "string",
      "description": "What this field represents",
      "minLength": 2
    },
    "field2": {
      "type": "integer",
      "minimum": 0
    }
  }
}
```

4.4 Exercise – JSON Schema for a Digital CV

Create a JSON Schema for a digital CV. The schema must validate a document that contains the following four sections:

Section	Type	Contents
identity	object	firstName, lastName, email, phone, dateOfBirth
education	array	Objects: degree, institution, year, mention
experience	array	Objects: jobTitle, company, startDate, endDate, current, description
skills	array	List of strings (skill names)

identity

Type: object. Required fields: firstName, lastName, email. Optional: phone, dateOfBirth.

- firstName and lastName must be strings.
- email must use the format: "email".
- dateOfBirth must use the format: "date" (YYYY-MM-DD).

education

Type: array of objects. At least one entry required (minItems: 1). Required fields per item: degree, institution, year.

- year must be an integer between 1950 and 2100.
- mention is optional and must be one of: Excellent, Very Good, Good, Satisfactory — use enum.

experience

Type: array of objects. Required fields per item: jobTitle, company, startDate.

- startDate and endDate must use the format: "date".
- current is a boolean indicating whether this is the current position.
- description is optional with a maxLength of 500 characters.

skills

Type: array of strings. At least one skill required (minItems: 1). Each skill must be a string of at least 2 characters.